

AI / ML INTERNSHIP

Assessment Projects

Choose one. Build it. Run it live at your interview.

How this works

This document contains **three independent problem statements**. Pick **any one** of them — they are equally weighted and there is no “right” choice. Build a working solution.

On the day of your interview you will **run your solution live**, on an input we provide, in front of the panel. Everything else in this brief exists to support that one moment.

Submission requirements, timeline, evaluation criteria and other logistics are covered in the accompanying email.

	Project	Core challenge
1	Document OCR — fine-tuned VLM with extraction agent	Fine-tune a small vision-language model and orchestrate it across a 10-page document.
2	PHI / PII de-identification gateway	Strip every patient identifier before a foundation LLM sees the text — without destroying clinical meaning.
3	Canonical medical record structuring pipeline	Turn a messy multi-document PDF into validated, queryable FHIR.

A note on scope

Each problem is deliberately open-ended. We describe **the outcome we need**, not the method. Choosing the architecture, the models, the data and the trade-offs is the assessment.

A narrow solution that works completely will always beat a broad one that half-works everywhere.

Contents

If the list below appears empty, right-click it and choose "Update Field" to populate page numbers.



PROJECT 1

Document OCR — fine-tuned VLM with extraction agent

P1.1 The problem

We receive documents as scanned PDFs and photographs — invoices, forms, reports, handwritten annotations, multi-column layouts, tables, stamps, skew, shadows. Generic OCR such as Tesseract returns character soup with no structure. Large commercial vision-language models return good results at a per-page cost we cannot sustain and with a data-residency profile our clients will not accept.

What we need

A small, self-hosted vision-language model that reads a document as well as a human would, and an agent around it capable of processing a full multi-page document unattended.

P1.2 Target outcome

A system where we hand over a **10-page PDF** and receive back **structured, ordered, accurate text and tables for all 10 pages**, with per-page confidence, produced by a model **you fine-tuned**, running on hardware **we can afford**.

P1.3 Hard requirements

Requirement	Constraint
Base model	Open-weights VLM, under 3B parameters. Report the exact count.
Fine-tuning method	LoRA or QLoRA. Full fine-tuning is out of budget and out of scope.
Input	A 10-page PDF or a folder of 10 page images. Mixed content, including at least one table and one non-trivial layout.
Output	Structured file (JSON / Markdown) preserving page order, reading order and table structure.
Orchestration	An agent layer around the model — see P1.5.
Runtime	Must complete 10 pages on a single consumer GPU. Report wall-clock time.

P1.4 The fine-tuning task

You choose the specialisation, but state it explicitly and build your dataset around it. Reasonable choices include:

- Handwritten or mixed handwriting-and-print recognition
- Table structure recognition with cell-level extraction
- A specific document class — invoices, lab reports, forms — with field-level extraction
- Layout-aware reading order for multi-column documents

Requirements

- Show a measured improvement over the base model on the same held-out set.
- If fine-tuning made performance **worse**, report that honestly. A well-diagnosed negative result scores higher than an unverifiable positive one.
- Publish your LoRA adapter weights (they are small) or a script that reproduces them.
- Document the full configuration: rank, alpha, target modules, dropout, learning-rate schedule, batch size, epochs, quantisation.

Starting points (not recommendations — evaluate for yourself)

Candidate base models: Qwen2-VL-2B, Qwen2.5-VL-3B, InternVL2-2B, MiniCPM-V, SmolVLM, Moondream2, GOT-OCR2.0, Florence-2, Donut.

Candidate datasets: FUNSD, CORD, SROIE, DocVQA, DocLayNet, PubTables-1M, IAM Handwriting, RVL-CDIP. Synthetic generation is legitimate and often the correct answer — if you generate data, describe the generator.

P1.5 The agent layer

This is the part most candidates under-build

A for page in pages: model(page) loop is **not an agent** and will not score.

At minimum the agent must:

- **Ingest and normalise** — PDF to images, DPI handling, skew and denoise decisions.
- **Route** — decide per page what to do. A dense table page and a signature page do not need the same treatment. Justify your routing policy.
- **Handle failure** — detect low-confidence or malformed output and act on it: retry with different prompt or parameters, fall back to a second method, or escalate. Demonstrate the retry actually firing on a genuinely bad page.

Assemble — stitch 10 pages into one coherent ordered document with continuity across page breaks. A table split across pages 4 and 5 should emerge as a single table.

- **Self-check** — validate its own output before returning it. A schema check at minimum; a semantic check is better.

Framework choice is yours — LangGraph, a plain Python state machine, anything else. Plain Python is fine and often better. We grade the design, not the dependency list.

P1.6 Evaluation

Baselines to beat: Tesseract OCR, and the un-fine-tuned base VLM. Report all three side by side.

Metric	Notes
Character Error Rate / Word Error Rate	Against your ground truth transcription.
Field-level precision, recall, F1	If performing structured extraction.
TEDS or cell-level F1	For tables.
Reading-order correctness	Kendall's tau, or a simple ordering accuracy.
Latency, peak VRAM, total runtime	Per page, and for the full 10-page document.

Test set: minimum 20 pages the model has never seen, hand-labelled by you, including at least three deliberately hard pages — poor scan, rotation, handwriting, dense table. Ship the test set with the repository.

P1.7 Stretch goals

- Confidence calibration — does the model's stated confidence actually predict its errors?
- Batched or paged inference that meaningfully reduces the 10-page wall-clock time.
- LoRA rank sweep compared against output quality and adapter size.

P1.8 Acceptance test

On the review call we will hand you a 10-page PDF you have never seen. It should run and produce sane output. Partial degradation is acceptable; a crash is not.

PROJECT 2

PHI / PII de-identification gateway**P2.1 The problem**

Healthcare clients want to use large foundation models on their clinical text. They cannot send identifiable patient data to a third-party API — legally, contractually, or ethically. Today this blocks the entire use case.

What we need

A de-identification gateway: a small model that sits in front of the foundation LLM, strips every identifier out of the text on the way in, and restores what is needed on the way out — without destroying the clinical meaning the LLM needs in order to be useful.

The real problem you are solving

Redact too little and you leak patient data. Redact too much and the downstream LLM becomes useless. Both are failures. We are assessing whether you understand that this tension — not entity tagging — is the actual problem.

P2.2 Target outcome

A service that accepts raw clinical text, returns de-identified text plus a secure mapping, forwards it to a foundation LLM, and rehydrates the response — demonstrated end to end on a real document.

P2.3 Hard requirements

Requirement	Constraint
Model	Trained or fine-tuned DL model / LLM, 1B parameters or fewer. Report the exact count.
Coverage	All 18 HIPAA Safe Harbor identifier categories — see P2.4.
Interface	Callable service: <code>deidentify(text) → (masked_text, mapping)</code> and <code>rehydrate(response, mapping) → text</code> .
Round trip	Must demonstrate the full loop: <code>raw → masked → foundation LLM → rehydrated answer</code> .

Requirement	Constraint
Data	Synthetic or properly licensed de-identification corpora only.

P2.4 Identifier scope

Handle all 18 HIPAA Safe Harbor categories:

#	Category	#	Category
1	Names	10	Account numbers
2	Geographic subdivisions smaller than a state	11	Certificate / licence numbers
3	All date elements except year (incl. DOB, admission, discharge, ages over 89)	12	Vehicle identifiers and plate numbers
4	Telephone numbers	13	Device identifiers and serial numbers
5	Fax numbers	14	URLs
6	Email addresses	15	IP addresses
7	Social security numbers	16	Biometric identifiers
8	Medical record numbers	17	Full-face photographs and comparable images
9	Health plan beneficiary numbers	18	Any other unique identifying number, characteristic or code

Not all of these appear in free text. State which you cover, which you deliberately excluded, and why.

P2.5 The hard parts

We are specifically testing these.

Challenge	Why it is hard
Ambiguity	"Dr. Parkinson diagnosed Parkinson's." "Mr. Wood was admitted to Wood Memorial." One token is PHI, the identical token is not. Rules fail here — which is why you are training a model.
Dates	Clinical reasoning often depends on intervals ("3 days post-op"). Naively removing all dates destroys utility. Consistent per-patient date shifting preserves intervals while de-identifying. Show that you considered this.
Ages over 89	A HIPAA-specific edge case. Handle it explicitly.
Recall asymmetry	A false negative is a data breach. A false positive is an inconvenience. Your loss function, thresholds and reporting should reflect that these are not equally bad.
Masking strategy	Redaction ([NAME]), consistent pseudonymisation (Patient_A), or surrogate generation (realistic fake names)? Each has different downstream effects. Pick

Challenge	Why it is hard
	one, justify it, ideally measure it.
Rehydration security	Where does the mapping live? How is it keyed? What happens if the LLM echoes a token that was never in the input?

P2.6 Suggested approaches

Model families at or under 1B: DistilBERT, BERT-base, RoBERTa-base, DeBERTa-v3-base/small, ModernBERT-base, ClinicalBERT / BioClinicalBERT, SmolLM2-360M, Qwen2.5-0.5B, Llama-3.2-1B.

Token classification with BIO tagging is the conventional framing. Generative masking with a small instruct model is a legitimate alternative — if you take that route, address hallucination and non-determinism explicitly.

Data sources: the i2b2 / n2c2 2014 de-identification challenge (requires a DUA), synthetic generation via Faker plus templated clinical notes, MTSamples, and augmentation of public discharge summaries.

P2.7 Evaluation

Baselines to beat: (a) regex-only, and (b) Microsoft Presidio or off-the-shelf spaCy NER. Report both.

Metric	Notes
Entity-level precision, recall, F1	Per category and overall.
Recall — the headline number	Report it prominently.
Leak rate	Percentage of documents containing at least one missed identifier. This is the number a compliance officer actually asks for.
Utility preservation	Choose a downstream task (summarisation, QA over the note) and measure a foundation LLM's performance on original versus de-identified text. Quantify what your masking costs.
Latency and throughput	p50 and p95.

Test set: minimum 50 held-out clinical notes with hand-verified identifier annotations. Include adversarial cases — nicknames, misspelled names, names embedded in narrative, and identifiers hiding in tables, headers and signature blocks.

P2.8 Stretch goals

- Consistent date shifting that preserves intervals within a patient record.
- Confidence thresholding with a human-review queue for uncertain spans.
- Re-identification risk scoring — could this note be re-identified from quasi-identifiers alone?
- Handling PHI in structured segments: headers, footers, tables, form fields.

P2.9 Acceptance test

On the review call we will hand you a clinical note. It should mask correctly, round-trip through a foundation LLM, and rehydrate. We will also read your masked output looking for leaks — assume we will find them if they exist.



PROJECT 3

Canonical medical record structuring pipeline

P3.1 The problem

A patient's medical record arrives as a single 200-page PDF: a fax dump containing a discharge summary, four lab reports, two radiology reports, an intake form, insurance paperwork, handwritten nursing notes, and thirty pages of duplicated fax cover sheets — in no reliable order, with no table of contents, originating from six different source systems each with its own layout.

Everything downstream — search, analytics, clinical decision support, billing — requires this to be structured, normalised and queryable. Today it is a PDF, which is to say it is nothing.

What we need

A pipeline that turns an unstructured multi-document PDF into a validated, standards-conformant, queryable clinical dataset.

Terminology note

HL7 FHIR — Fast Healthcare Interoperability Resources — release R4 or R5.

P3.2 Target outcome

Feed in one messy multi-page PDF. Get out: correctly segmented sub-documents, each classified, each parsed, all clinical facts extracted, coded against standard terminologies, emitted as valid FHIR resources, and persisted in a queryable store — with full provenance from every field back to its source page.

P3.3 Required pipeline stages

Stage 1 — Page-wise grouping and document segmentation

The hardest and most under-appreciated stage. A 200-page bundle is not one document; it is fifteen documents concatenated.

- Classify each page by type: lab report, discharge summary, radiology, intake form, insurance, cover sheet, blank, duplicate.
- Detect **document boundaries** — where one report ends and the next begins. Headers, footers, “Page 1 of 4” markers, layout shifts, letterhead changes and semantic discontinuity are all signal.
- Group contiguous pages into logical documents.
- Detect and flag duplicates and blank pages.

Report boundary detection accuracy separately

If this stage is 70% accurate, nothing downstream can exceed 70%. We want to see that you understand and quantify that ceiling.

Stage 2 — Content extraction

Per document type, extract raw content: text, tables, key-value pairs, form fields. Reusing an existing OCR or VLM stack is fine — extraction quality is not the primary focus of this project. Structuring is.

Stage 3 — Clinical entity extraction

Extract the clinical facts: patient demographics, encounters, conditions and diagnoses, medications (with dose, route, frequency), allergies, procedures, laboratory results (with value, unit, reference range, abnormal flag), vitals, immunisations, providers, and dates.

Stage 4 — Terminology normalisation

Map free text to standard codes. This is what “canonical” means.

Domain	Target terminology
Diagnoses and conditions	ICD-10-CM and / or SNOMED CT
Laboratory tests and observations	LOINC
Medications	RxNorm
Procedures	CPT and / or SNOMED CT
Units of measure	UCUM

Handle synonyms, abbreviations (“MI”, “HTN”, “SOB”), misspellings, brand versus generic drug names, unit conversion and harmonisation (mg/dL ↔ mmol/L), and the many cases where no confident mapping exists.

Unmapped is acceptable. Silently wrong is not.

Every code you emit must carry a confidence score. A wrong code with no confidence signal is worse than no code at all.

Stage 5 — FHIR resource construction

Emit valid FHIR R4 resources with correct references between them. Minimum set: Patient, Encounter, Condition, Observation, MedicationStatement (or MedicationRequest), AllergyIntolerance, Procedure, DiagnosticReport, DocumentReference. Bundle them.

Resources must **validate** against the FHIR specification. Use the official validator or the fhir.resources library, and ship the validation output.

Stage 6 — Persistence

Store the result in something queryable: a normalised relational schema, an indexed document store, or a FHIR server such as HAPI FHIR. Your call — justify it. Demonstrate with at least five real clinical queries, for example:

- All abnormal laboratory results for this patient, in date order.
- Full medication list with active / discontinued status.
- Every condition coded to ICD-10, together with the source page it came from.

Stage 7 — Provenance and confidence

Non-negotiable. Every structured field must trace back to its source document, page number, and ideally text span. Every field carries a confidence score. Low-confidence fields route to a review queue rather than being written silently as fact.

In clinical software, a wrong value with no audit trail is worse than a missing one.

P3.4 Hard requirements

Requirement	Constraint
Input	Multi-document PDF, 30 pages or more, containing at least 4 distinct document types.
Output	Validated FHIR R4 bundle plus a populated queryable store.
Deduplication	The same patient or fact appearing across multiple source documents must be reconciled, not duplicated.
Conflict handling	When two documents disagree — different DOB, different medication dose — you must have a documented resolution policy.
Provenance	Every field traceable to its source page.
Data	Synthetic only. Synthea is the obvious generator.

P3.5 Suggested resources

- **Synthea** — synthetic patient generator; produces FHIR you can use as ground truth.
- **MTSamples** — public transcribed medical reports.
- **MIMIC-IV** (requires credentialing) or its open demo subset.
- **UMLS, RxNav, LOINC, Athena (OHDSI)** — terminology sources; most are free with registration.
- **HAPI FHIR, fhir.resources**, and the official HL7 FHIR validator.
- **cTAKES, MedSpaCy, scispaCy, QuickUMLS** — clinical NLP baselines worth benchmarking against.

P3.6 Evaluation

Baseline to beat: a naive pipeline — treat the PDF as one document, regex-extract, no terminology mapping. Show the delta.

Metric	Notes
Page classification accuracy	Per class, plus a confusion matrix.
Document boundary detection	Precision and recall on boundaries.
Entity extraction F1	Per entity type.
Terminology mapping accuracy	On a hand-verified sample of at least 100 mappings, plus coverage (percentage mapped at all).
FHIR validation pass rate	Target 100%. Report every failure.
Deduplication precision and recall	Across source documents.
End-to-end recovery	Given a Synthea record you rendered to PDF, how much of the original structured truth did you recover? This is the single most informative number in the project.

P3.7 Stretch goals

- Longitudinal timeline construction across encounters.
- Cross-document patient identity resolution — same patient, name spelled differently.
- Human-in-the-loop review interface for low-confidence fields.
- Incremental ingestion — a new document arrives for an existing patient and merges correctly.

P3.8 Acceptance test

On the review call we will hand you a multi-document synthetic PDF. It should segment, extract, normalise, validate and persist. We will then query the store live.

4. Timeline

Day	Milestone
0	Assignment issued. Clarification window opens.
2	Approach note due — one page: chosen models, datasets, architecture sketch, risks. We respond within 24 hours. This is a checkpoint, not a gate; it exists to stop you spending six days on a dead end.
5	Optional mid-point check-in, 15 minutes. Optional but recommended.
7	Final submission — repository, report, video.
8 – 10	Interview: demo, code walkthrough, live acceptance test, technical discussion.

Adjust for candidate availability. If you are working full-time alongside this assessment, tell us — we will extend rather than penalise you for it.

5. Guidance to candidates

Advice	Why
Scope aggressively.	A narrow pipeline that works completely beats a broad one that half-works everywhere. If you must choose, choose depth on the core requirement.
Build the end-to-end skeleton on day one, with stub components.	Then improve the weakest link repeatedly. Candidates who spend five days perfecting the model and one day on integration reliably submit something that does not run.
Measure early.	You cannot improve what you have not measured. Building the evaluation harness on day six means your reported numbers are guesses.
Write FAILURES.md as you go.	It is graded, and the specific version written during development is worth far more than the vague one written the night before.
Ask questions.	The brief is deliberately ambiguous in places, mirroring real requirements. A good clarifying question is a positive signal. Silently guessing on something material is not.

6. Clarifications policy

Post questions to the clarifications channel given in the covering email. Answers are shared with all candidates.

We will answer questions about **requirements and constraints**. We will not answer questions about **approach** — “should I use Qwen or InternVL” is the assessment itself, and we will simply tell you that the choice is yours.

End of assessment brief

